

CHAPTER 3

REQUIREMENTS & ANALYSIS

The current manual or semi-automated methods of renting and managing cars lead to inefficiencies, booking conflicts, and lack of real-time availability updates. Customers face issues like non-transparent pricing, delayed confirmations, and poor tracking of bookings, while rental companies struggle with inventory visibility, maintenance tracking, and customer data management.

3.1 PROBLEM DEFINITION

Traditional vehicle rental systems often involve lengthy paperwork, limited vehicle options, location-based restrictions, and rigid rental policies that do not cater to the dynamic and short-term mobility needs of modern users. Additionally, conventional rental agencies usually operate with a centralized inventory, resulting in higher operational costs and restricted availability during peak periods or in remote areas.

In contrast, the demand for flexible, accessible, and cost-effective self-drive transportation has grown significantly, especially among urban dwellers, tourists, and working professionals. However, despite the availability of vehicles privately owned by individuals, there is no widely adopted, user-friendly digital platform that connects these vehicle owners with potential renters in a secure and structured environment.

This project aims to solve the inefficiencies of existing rental models by developing a peer-to-peer Self-Drive Car Booking System. The proposed platform allows car owners to monetize their idle vehicles by listing them for rent, while enabling users to browse, select, and book cars based on location, availability, and pricing. It eliminates the traditional middleman, reduces rental costs, and provides more personalized and varied vehicle options.

The system also introduces features such as secure user registration and verification, detailed car listings, geolocation-based search, rating and review mechanisms, and an integrated payment gateway. By leveraging modern web technologies and a decentralized rental model, the Self-Drive Car Booking System enhances trust, improves vehicle utilization, and ensures

convenience and flexibility for both parties. It represents a shift towards a community-driven car-sharing economy that aligns with evolving transportation preferences and digital lifestyles.

3.2 REQUIREMENTS SPECIFICATION

The proposed Self Drive Car Booking System is designed to fulfill both functional and non-functional requirements to ensure a seamless and efficient Self Drive Car experience for both customers and administrators. Functionally, the system will support user registration and secure authentication. Customers will be able to view available cars, filter them by criteria such as location, rental dates, and car category, and then proceed to book a car of their choice. Each booking will capture the duration, calculate the total rental cost based on the daily rate, and track the booking status (e.g., confirmed, cancelled, completed). Administrators will be equipped with tools to manage the car inventory, including the ability to add, update, or remove car listings. The system will also allow administrators to view booking details, manage customer information, and monitor car availability. Email notifications will be triggered for key booking actions to ensure transparency and timely communication with customers.

3.2.1 FUNCTIONAL REQUIREMENTS

- User registration and login with identity verification
- Car owner dashboard to manage listings
- Renter dashboard to browse and book cars
- Booking management and scheduling system
- Integrated payment gateway
- Rating and review module
- Admin panel for system monitoring

3.2.2 NON-FUNCTIONAL REQUIREMENTS

- Secure authentication and encrypted data handling
- Mobile responsiveness
- High availability and scalability
- User-friendly interface
- GDPR and data compliance

3.3 PLANNING AND SCHEDULING

The development of the Self-Drive Car Booking System was executed in a phased and systematic manner to ensure clarity, collaboration, and timely completion. The project began with a comprehensive requirement analysis phase, during which user needs, functional requirements, and system goals were clearly defined. This was followed by the design phase, where UI wireframes, ER diagrams, and user journey maps were prepared to guide development.

Once the design blueprint was established, the project was divided into backend and frontend development modules. The backend was powered by Supabase with PostgreSQL, handling user authentication, vehicle listing management, booking records, and secure data storage. Simultaneously, the frontend was developed using React and TypeScript to deliver a responsive, mobile-friendly user interface for both car owners and renters.

Development followed an agile sprint model, with each module—such as user registration, vehicle listing, booking functionality, and dashboard views—built, tested, and integrated iteratively. Weekly stand-ups were conducted to assess progress, manage tasks, and identify bottlenecks early. Testing and validation were prioritized during every phase to ensure system stability, security, and data consistency.

The final stages included full system integration, user testing, deployment to a live environment, and feedback collection to refine the user experience. This phased approach enabled smooth collaboration, effective problem resolution, and ensured delivery of a feature-complete platform within the allocated timeline.

Table 3.1: Project Plan

Phase	Start Date	End Date	Duration
Inception Phase	20/06/2025	03/07/2025	14
Elaboration Phase	10/07/2025	27/07/2025	18
Construction Phase	26/08/2025	20/09/2025	26
Transition Phase	24/09/2025	25/10/2025	32

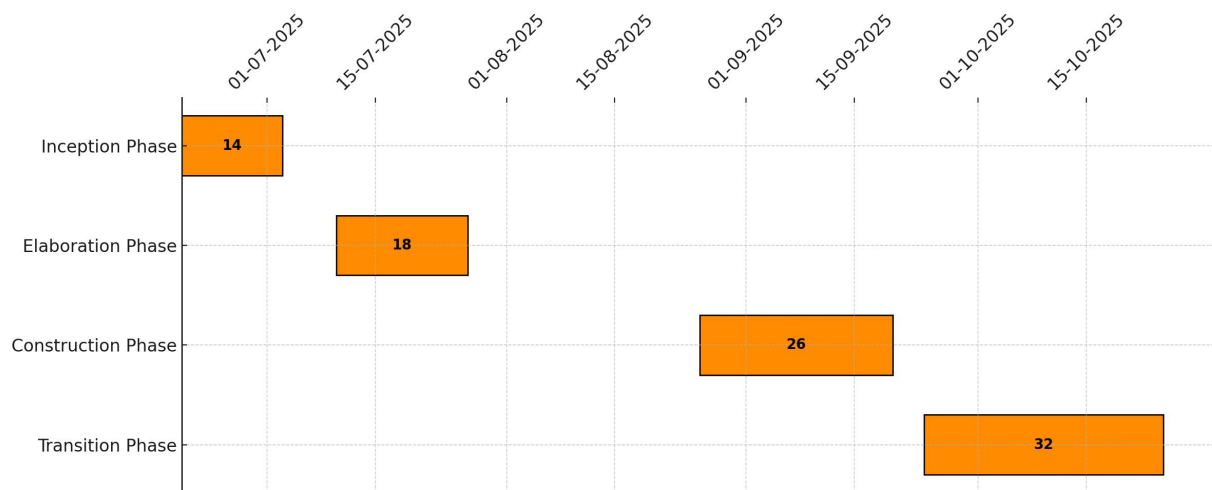


Figure 3.1: Gantt Chart for Project Plan

Table 3.2: Project Plan Phases

Phase	Description	Milestone
Inception Phase	Define the project vision, scope, feasibility, and success criteria. Conduct high-level risk analysis to ensure project viability.	Approved project charter Feasibility report Initial technology and tool selection
Elaboration Phase	Design the system architecture, define user roles and interactions, and prepare detailed module requirements.	UI/UX wireframes Database schema design Core module prototypes Updated risk analysis
Construction Phase	Develop full application modules, including frontend and backend integration. project creation	Fully functional beta system Test reports Real-time data tracking and role-based access
Transition Phase	Deploy the platform, perform validation and user testing. Prepare training documentation and handover system for ongoing maintenance and monitoring.	Final deployment Admin and user manuals Production-ready system Project sign-off

Table 3.3: Project Schedule

Task Name	Start Date	End Date	Duration
Architectural Prototype (Design)	14/07/2025	05/08/2025	23
Construction	07/09/2025	25/09/2025	19
Transition (Implementation and Training)	04/10/2025	28/10/2025	25

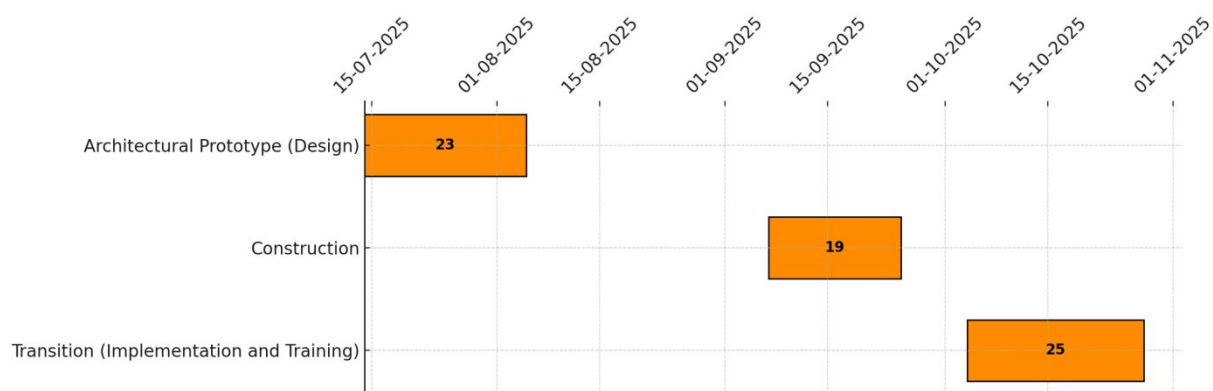


Figure 3.2: Gantt Chart for Project Schedule

3.4 SOFTWARE AND HARDWARE REQUIREMENTS

The successful development and deployment of the Self Drive Car Booking System rely on specific software and hardware components that collectively support application functionality, performance, and security. On the software side, the system is built using TypeScript as the primary programming language for the frontend, ensuring enhanced code readability, maintainability, and error reduction. React is used as the UI framework to create a responsive and interactive user interface. The backend is powered by Supabase, a modern backend-as-a-service platform built on PostgreSQL, which provides powerful database management, real-time data handling, and built-in user authentication. Supabase also facilitates RESTful APIs and seamless integration with modern frontend stacks, reducing the need for

manual server configuration. Development tools such as Visual Studio Code, Node.js, and npm are essential for code editing, building, and managing project dependencies.

3.4.1 SOFTWARE REQUIREMENTS

Frontend: HTML, CSS, JavaScript (React)

Backend: Supabase (PostgreSQL, Auth, Storage)

Tools: VS Code, GitHub

Browser: Chrome / Firefox

3.4.2 HARDWARE REQUIREMENTS

Processor: Intel i3 or higher

RAM: Minimum 4 GB

Storage: 250 GB HDD/SSD

Operating System: Windows 10 / Linux / macOS

3.5 PRELIMINARY PRODUCTION

The Self-Drive Car Booking System is a secure and user-friendly web application aimed at modernizing the way individuals rent vehicles for self-driven, short-term use. The platform enables seamless interaction between private car owners and renters, creating a peer-to-peer car-sharing ecosystem. Designed with a focus on convenience, transparency, and safety, the system empowers car owners to list their vehicles and earn passive income while providing renters with a flexible alternative to traditional Self Drive Car services. The core functionality of the system centers around vehicle listing, discovery, and booking. Car owners can register on the platform, verify their identity, and create detailed listings by entering vehicle information such as make, model, year, condition, rental rate, and availability. Renters, on the other hand, can browse available vehicles based on location, filter options, and rental period preferences. Once a suitable vehicle is found, they can securely book it through the platform's integrated booking and payment system. The platform also includes features like real-time availability tracking, user authentication, review and rating systems, and responsive design to ensure accessibility across devices. By ensuring a verified user base and transparent rental process, the system fosters trust between owners and renters. This innovative approach to car sharing is

particularly beneficial in urban areas, where users seek cost-effective, short-term mobility solutions without the burden of car ownership.

3.6 CONCEPTUAL MODEL

3.6.1 ENTITY RELATIONSHIP DIAGRAM

An Entity-Relationship Diagram (ERD) is a vital tool used in the design phase of a database-driven system. It provides a visual representation of entities within the system and the relationships among them. For the Self-Drive Booking System, the ERD outlines the core entities such as users, vehicles, bookings, and payments, and their interactions.

ENTITY

User: Attributes could include Id, name, email, password, role(owner or renter)

Vehicle: Attributes could include id, owner_id, make, model, year, registration_number, price_per_day, availability_status, location, image_url

Booking: Attributes could include id, vehicle_id, renter_id, start_date, end_date, total_amount, status

Payment: Attributes could include id, vehicle_id, renter_id, start_date, end_date, total_amount, status

RELATIONSHIP:

User to Vehicle (1:N): A single car owner (user) can list multiple vehicles.

Example: One user can list both a sedan and an SUV for rent.

Vehicle to Booking (1:N): A single vehicle can be booked multiple times over different periods.

Example: A car may be rented by multiple users on different dates.

User to Booking (1:N): A renter (user) can make multiple bookings for different vehicles.

Example: A user can book a car for a weekend trip and another for business travel.

Booking to Payment (1:1): Each booking has one associated payment record.

Example: When a booking is confirmed, a payment transaction is processed and recorded.

ER DIAGRAM:

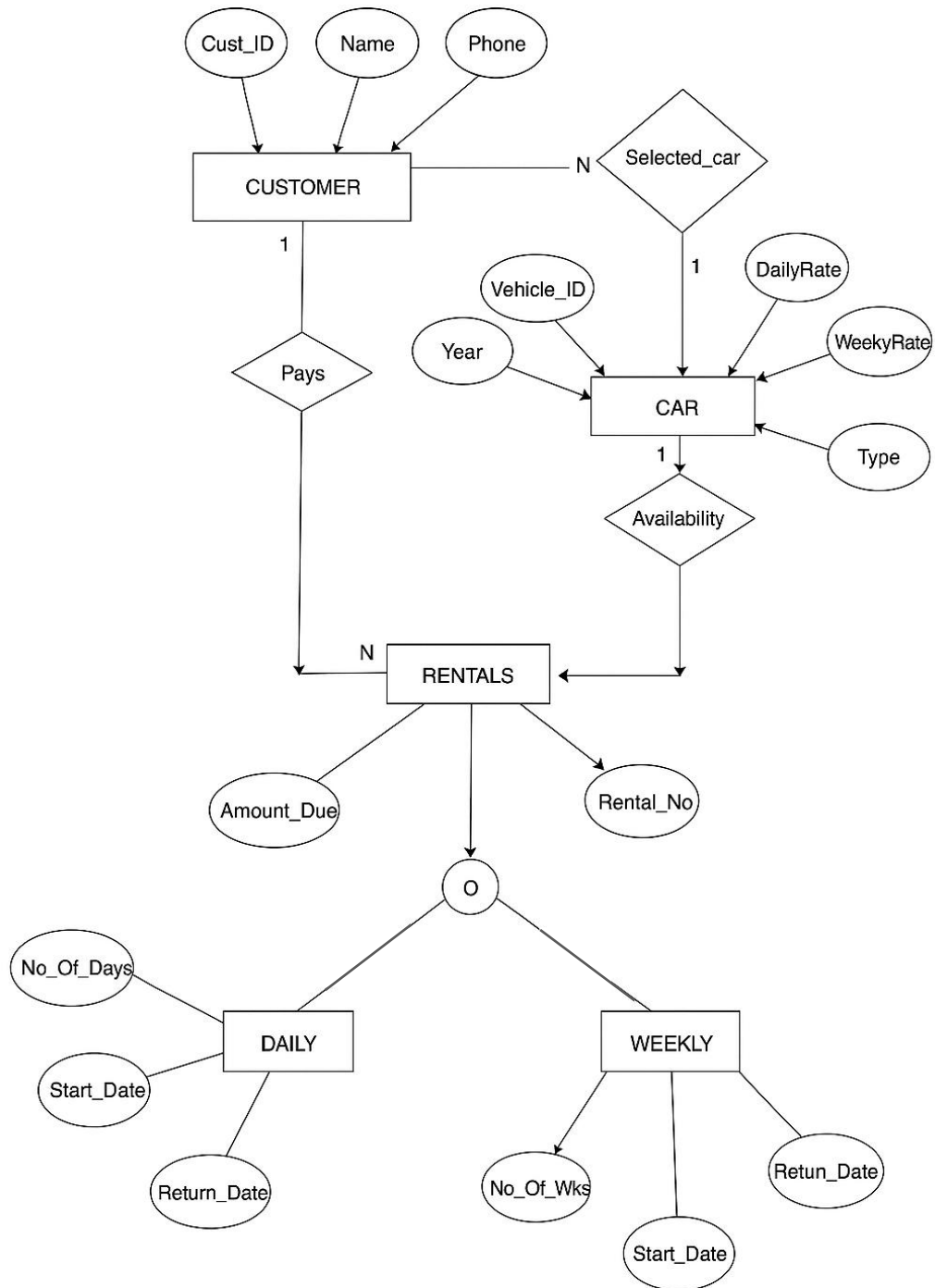


Figure 3.3 ER Diagram